

```
1 // Problem 1: Given the number of vertices and edges of a graph, the task is to represent the adjacency
2 // directed graph.
3 // Input: V=3, edges[][]={{0,1},{1,2},{2,0}}
4 // Output:
5 // 0 -> 1
6 // 1 -> 2
7 // 2 -> 0
8 // Input: V=4, edges[][]={{0,1},{1,2},{1,3},{2,3},{3,0}}
9 // Output:
10 // 0 -> 1
11 // 1 -> 2 3
12 // 2 -> 3
13 // 3 -> 0
14
15 #include <stdio.h>
16 #include <stdlib.h>
17
18 void main()
19 {
20     int v, e;
21
22     printf("Enter no. of vertices");
23     scanf("%d", &v);
24
25     printf("Enter no. of edges");
26     scanf("%d", &e);
27     int arr[e][2];
28
29     for (int i = 0; i < e; i++)
30     {
31         printf("Enter endpoint of edges");
32         for (int j = 0; j < 2; j++)
33         {
34             printf("enter edge[%d][%d]", i, j);
35             scanf("%d", &arr[i][j]);
36         }
37     }
38     for (int i = 0; i < v; i++)
39     {
40         printf("%d -> ", i);
41         for (int j = 0; j < e; j++)
42         {
43             if (arr[j][0] == i)
44             {
45                 printf("%d", arr[j][1]);
46             }
47         }
48         printf("\n");
49     }
50 }
```

```

1 // Problem 2: Write a program to perform the following basic graph-related operations:
2 // 1. Represent a graph using adjacency matrix.
3 // 2. Represent a graph using adjacency list.
4 // 3. Find the degree of each vertex in the graph.
5 // 4. Determine whether the graph is simple, complete, or connected.
6 // 5. Display total number of vertices and edges.
7 // Input: V = 5, edges [][] = {(0, 1), (0,2), (1,2), (2,3), (3,4)}
8 // Output:
9 // 1. Adjacency Matrix
10 // 2. Adjacency List
11 // 3. Degree of each vertex
12 // 4. Graph properties
13 // 5. Total vertices and edges
14 #include <stdio.h>
15 #include <stdlib.h>
16
17 void main()
18 {
19     int v, e;
20
21     printf("Enter no. of vertices");
22     scanf("%d", &v);
23
24     printf("Enter no. of edges");
25     scanf("%d", &e);
26
27     int add[v][v];
28
29     for (int i = 0; i < v; i++)
30     {
31         for (int j = 0; j < v; j++)
32         {
33             add[i][j]=0;
34         }
35     }
36     int n1,n2;
37
38     for (int i = 0; i < e; i++)
39     {
40         printf("Enter Edges");
41         scanf("%d %d",&n1,&n2);
42         add[n1][n2]=1;
43         add[n2][n1]=1;
44     }
45
46     for (int i = 0; i < v; i++){
47         printf("%d -> ", i);
48         for (int j = 0; j < v; j++)
49         {
50             if(add[i][j]==1){
51                 printf("%d ",j);
52             }
53         }
54         printf("\n");
55     }
56     printf("Adjacency Matrix \n");
57     for (int i = 0; i < v; i++){
58         for (int j = 0; j < v; j++)
59         {
60             printf("%d ",add[i][j]);
61         }
62         printf("\n");
63     }
64 }
65

```

```
66 //0 1 0 0
67 //1 0 1 1
68 //0 1 0 1
69 //0 1 1 0
```

```
1 // Objective: To implement Breadth First Search traversal for a graph.
2 // Problem: Perform a Breadth First Search (BFS) traversal starting from vertex 0.
3 // Input: adj[][] = [[1,2], [0,2,3], [0,1,4], [1,4], [2,3]]
4 // Output:
5 // [0, 1, 2, 3, 4]
6 #include<stdio.h>
7 void main(){
8
9     int adj[5][5]={
10         {0,1,1,0,0},
11         {1,0,1,1,0},
12         {1,1,0,0,1},
13         {0,1,0,0,1},
14         {0,0,1,1,0}
15     };
16     int visited[5]={0};
17     int queue[5];
18     int f=0,r=0;
19
20     visited[0]=1;
21     queue[r++]=0;
22
23     while(f<r){
24         int v=queue[f++];
25
26         printf("%d ",v);
27
28         for(int i=0;i<5;i++){
29             if(adj[v][i] && !visited[i]){
30                 visited[i]=1;
31                 queue[r++]=i;
32             }
33         }
34     }
35 }
```

```
1 // Objective: To implement Depth First Search traversal for a graph.
2 // Problem: Write a program to perform DFS traversal starting from a given source vertex.
3 // Input: adj[][] = [[1,2], [0,2], [0,1,3,4], [2], [2]]
4 // Output:
5 // [0, 1, 2, 3, 4]
6 #include <stdio.h>
7
8 #define V 5
9
10 int graph[V][V] = {
11     {0,1,1,0,0},
12     {1,0,1,0,0},
13     {1,1,0,1,1},
14     {0,0,1,0,0},
15     {0,0,1,0,0}
16 };
17
18 int visited[V];
19
20 void DFS(int v){
21     visited[v] = 1;
22     printf("%d ", v);
23
24     for(int i=0; i<V; i++){
25         if(graph[v][i] && !visited[i]){
26             DFS(i);
27         }
28     }
29 }
30
31 int main(){
32     int source = 0;
33
34     printf("DFS Traversal: ");
35     DFS(source);
36
37     return 0;
38 }
```

```
1 // Objective: To implement Depth First Search traversal for a graph.
2 // Problem: Write a program to perform DFS traversal starting from a given source vertex.
3 // Input: adj[][] = [[1,2], [0,2], [0,1,3,4], [2], [2]]
4 // Output:
5 // [0, 1, 2, 3, 4]
6 #include <stdio.h>
7
8 int adj[5][5] = {
9     {0,1,1,0,0},
10    {1,0,1,1,0},
11    {1,1,0,0,1},
12    {0,1,0,0,1},
13    {0,0,1,1,0}
14 };
15
16 int visited[5] = {0};
17
18 void DFS(int v) {
19     visited[v] = 1;
20     printf("%d ", v);
21
22     for(int i = 0; i < 5; i++) {
23         if(adj[v][i] && !visited[i]) {
24             DFS(i);
25         }
26     }
27 }
28
29 int main() {
30     DFS(0);
31     return 0;
32 }
```

```
1 // Objective: To understand the basic properties and structure of trees using graph operations.
2 // Problem 1: Write a program to perform the following operations on a tree:
3 // 1. Identify all pendent (leaf) vertices.
4 // 2. Find the degree of each vertex.
5 // 3. Find distance between vertices.
6 // 4. Find eccentricity of each vertex
7 // 5. Find the center of the tree.
8 // 6. Verify whether the given graph is a tree or not.
9 // Input: V = 6, edges [][] = {(0,1), (1,2), (1,3), (3,4), (3,5)}
10 // Output:
11 // 1. List of pendent vertices
12 // 2. Degree of each vertex
13 // 3. Distance matrix
14 // 4. Eccentricity of each vertex
15 // 5. Center of the tree
16 // 6. Confirmation whether the graph is a tree
17
18 #include <stdio.h>
19 #include <stdlib.h>
20
21 #define MAX 20
22
23 int adj[MAX][MAX];
24 int dist[MAX];
25 int queue[MAX];
26 int visited[MAX];
27
28 void bfs(int V, int start)
29 {
30     int front = 0, rear = 0;
31
32     for (int i = 0; i < V; i++)
33     {
34         visited[i] = 0;
35         dist[i] = -1;
36     }
37
38     visited[start] = 1;
39     dist[start] = 0;
40     queue[rear++] = start;
41
42     while (front < rear)
43     {
44         int u = queue[front++];
45
46         for (int v = 0; v < V; v++)
47         {
48             if (adj[u][v] == 1 && !visited[v])
49             {
50                 visited[v] = 1;
51                 dist[v] = dist[u] + 1;
52                 queue[rear++] = v;
53             }
54         }
55     }
56 }
57
58 void dfs(int v, int V) {
59     visited[v] = 1;
60     for(int i = 0; i < V; i++) {
61         if(adj[v][i] == 1 && !visited[i]) {
62             dfs(i, V);
63         }
64     }
65 }
```

```

66
67 void main()
68 {
69     int V, E;
70
71     printf("Enter number of vertices: ");
72     scanf("%d", &V);
73
74     printf("Enter number of edges: ");
75     scanf("%d", &E);
76
77     for (int i = 0; i < V; i++)
78     {
79         for (int j = 0; j < V; j++)
80         {
81             adj[i][j] = 0;
82         }
83     }
84
85     printf("Enter edges (u v):\n");
86     for (int i = 0; i < E; i++)
87     {
88         int u, v;
89         scanf("%d %d", &u, &v);
90         adj[u][v] = 1;
91         adj[v][u] = 1;
92     }
93
94     // --- 1. Identify all pendent (leaf) vertices ---
95     printf("\n--- 1. Pendent Vertices ---\n");
96     for (int i = 0; i < V; i++){
97         int cnt = 0;
98         for (int j = 0; j < V; j++){
99             if(adj[i][j] == 1) cnt++;
100         }
101         if(cnt == 1){
102             printf("%d is Pendent Vertex\n", i);
103         }
104     }
105
106     // --- 2. Find the degree of each vertex ---
107     printf("\n--- 2. Degree of Each Vertex ---\n");
108     for (int i = 0; i < V; i++){
109         int cnt = 0;
110         for (int j = 0; j < V; j++){
111             if(adj[i][j] == 1) cnt++;
112         }
113         printf("Vertex %d has %d degree\n", i, cnt);
114     }
115
116     // --- 3. Find distance between vertices ---
117     printf("\n--- 3. Distance Matrix ---\n ");
118     for (int i = 0; i < V; i++)
119         printf("%d ", i);
120     printf("\n");
121     for (int i = 0; i < V; i++)
122     {
123         bfs(V, i);
124         printf("%d: ", i);
125         for (int j = 0; j < V; j++)
126             printf("%d ", dist[j]);
127         printf("\n");
128     }
129
130     // --- 4. Find eccentricity of each vertex ---
131     printf("\n--- 4. Eccentricity of Each Vertex ---\n");
132     int ecc[MAX];

```



```

133     for (int i = 0; i < V; i++)
134     {
135         bfs(V, i);
136         int maxDist = 0;
137         for (int j = 0; j < V; j++)
138         {
139             if (dist[j] > maxDist)
140                 maxDist = dist[j];
141         }
142         ecc[i] = maxDist;
143         printf("Vertex %d = %d\n", i, maxDist);
144     }
145
146     // --- 5. Find the center of the tree ---
147     printf("\n--- 5. Center of the Tree ---\n");
148     int minEcc = 9999;
149     for (int i = 0; i < V; i++) {
150         if(ecc[i] < minEcc) minEcc = ecc[i];
151     }
152     for (int i = 0; i < V; i++) {
153         if(ecc[i] == minEcc) {
154             printf("Vertex %d is center\n", i);
155         }
156     }
157
158     // --- 6. Verify whether the given graph is a tree or not ---
159     printf("\n--- 6. Verify Graph is a Tree ---\n");
160     for(int i = 0; i < V; i++) visited[i] = 0;
161
162     int connected = 1;
163     if(V > 0) {
164         dfs(0, V);
165         for(int i = 0; i < V; i++) {
166             if(!visited[i]) {
167                 connected = 0;
168                 break;
169             }
170         }
171     }
172
173     if(connected && E == V - 1) {
174         printf("The graph is a tree.\n");
175     } else {
176         printf("The graph is NOT a tree.\n");
177     }
178 }

```

```

1 // Objective: To analyze the structural properties of a connected graph using spanning trees, branch
2 // Problem 1: In a social network of N people, some of them are directly connected as friends. If pe
3 // Your task is to determine the total number of friend circles.
4 // Input:
5 // 4 // total N people
6 // 1 1 0 0 // adj. Matrix
7 // 1 1 0 0
8 // 0 0 1 1
9 // 0 0 1 1
10 // Output:
11 // 2
12 #include <stdio.h>
13
14 #define MAX 20
15
16 int adj[MAX][MAX];
17 int queue[MAX];
18 int visited[MAX];
19
20
21 void bfs(int V, int start)
22 {
23     int front = 0, rear = 0;
24
25     visited[start] = 1;
26     queue[rear++] = start;
27
28     while (front < rear)
29     {
30         int u = queue[front++];
31
32         for (int v = 0; v < V; v++)
33         {
34             if (adj[u][v] == 1 && !visited[v])
35             {
36                 visited[v] = 1;
37                 queue[rear++] = v;
38             }
39         }
40     }
41 }
42
43 void main()
44 {
45     int V;
46
47     printf("Enter number of peoples: ");
48     scanf("%d", &V);
49
50     for (int i = 0; i < V; i++)
51     {
52         for (int j = 0; j < V; j++)
53         {
54             scanf("%d", &adj[i][j]);
55         }
56     }
57
58     for(int i=0;i<V;i++){
59         visited[i]=0;
60     }
61
62     int cnt=0;
63
64     for(int i=0;i<V;i++){
65         if(visited[i]==0){

```

```
66         bfs(V,i);
67         cnt++;
68     }
69 }
70 printf("Total number of Circles : %d",cnt);
71 }
```

```
1 // Objective: To analyze the structural properties of a connected graph using spanning trees, branch
2 // Problem 2: Write a program to perform the following operations on a connected graph:
3 // 1. Construct a spanning tree of the graph.
4 // 2. Identify all branches and chords.
5 // 3. Find all fundamental circuits
6 // 4. Calculate: Rank and Nullity of the graph
7 // 5. Verify that: Number of branches = Rank and Number of chords = Nullity.
8 // Input:
9 // Enter number of vertices: 5
10 // Enter number of edges: 6
11 // edges[][] = {(0,1),(0,2),(1,2),(1,3),(2,4),(3,4)}
12 // Output:
13 // Branches:
14 // (0,1)
15 // (0,2)
16 // (1,3)
17 // (2,4)
18 // Chords:
19 // (1,2)
20 // (3,4)
21 // Fundamental Circuits: C1 = {(0,1),(0,2),(1,2)} C2 = {(1,3),(2,4),(3,4)}
22 // Rank of Graph = 4
23 // Nullity of Graph = 2
```

```
1 // Objective: To identify articulation points in an undirected graph and determine connectivity and
2 // Problem: Write a program to find all cut-vertices of a graph.
3 // Input: Connected graph with vertices and edges.
4 // Output:
5 // List of articulation points.
6 #include <stdio.h>
7 #include <stdbool.h>
8
9 #define MAX 100
10 int graph[MAX][MAX];
11 int visited[MAX], disc[MAX], low[MAX], parent[MAX];
12 bool ap[MAX];
13 int V, E;
14 int time = 0;
15
16 void DFS(int u)
17 {
18     int children = 0;
19     visited[u] = 1;
20     disc[u] = low[u] = ++time;
21
22     for (int v = 0; v < V; v++)
23     {
24         if (graph[u][v])
25         {
26             if (!visited[v])
27             {
28                 children++;
29                 parent[v] = u;
30                 DFS(v);
31
32                 if (low[v] < low[u])
33                     low[u] = low[v];
34
35                 if (parent[u] == -1 && children > 1)
36                     ap[u] = true;
37
38                 if (parent[u] != -1 && low[v] >= disc[u])
39                     ap[u] = true;
40                 else if (v != parent[u])
41                 {
42                     if (disc[v] < low[u])
43                         low[u] = disc[v];
44                 }
45             }
46         }
47     }
48 }
49
50 int main()
51 {
52     printf("Enter number of vertices: ");
53     scanf("%d", &V);
54
55     printf("Enter number of edges: ");
56     scanf("%d", &E);
57
58     for (int i = 0; i < V; i++)
59     {
60         visited[i] = 0;
61         parent[i] = -1;
62         ap[i] = false;
63
64         for (int j = 0; j < V; j++)
65             graph[i][j] = 0;
```

```

66
67     printf("Enter edges (u v):\n");
68     for (int i = 0; i < E; i++)
69     {
70         int u, v;
71         scanf("%d %d", &u, &v);
72
73         graph[u][v] = 1;
74         graph[v][u] = 1;
75     }
76
77     for (int i = 0; i < V; i++)
78     {
79         if (!visited[i])
80             DFS(i);
81     }
82
83     printf("\nArticulation Vertices :\n");
84
85     int found = 0;
86     for (int i = 0; i < V; i++)
87     {
88         if (ap[i])
89         {
90             printf("%d ", i);
91             found = 1;
92         }
93     }
94
95     if (!found)
96         printf("No Articulation Vertices");
97
98     printf("\n");
99
100     return 0;
101 }

```

```
1 // Objective: Determine connectivity and separability of a graph
2 // Problem: Write a program to:
3 // 1. Find vertex connectivity
4 // 2. Find edge connectivity
5 // 3. Construct fundamental cut-sets
6 // 4. Determine whether graph is separable or non-separable
7 // Input: Undirected graph.
8 // Output: Connectivity values and graph type
9 #include<stdio.h>
10 #define V 6
11 #define chord 3
12
13 void DFS(int graph[V][V], int visited[],int v){
14
15     visited[v]=1;
16     int i=0;
17     for(int i=0;i<V;i++){
18         if(graph[v][i] && !visited[i]){
19             DFS(graph, visited, i);
20         }
21     }
22 }
23
24 int isConnected(int graph[V][V]){
25     int visited[V] = {0};
26     int i=0;
27
28     DFS(graph, visited,0);
29
30     for(int i=0;i<V;i++){
31         if(!visited[i])
32             return 0;
33     }
34     return 1;
35 }
36
37 int vertexConnectivity(int graph[V][V])
38 {
39     int temp[V][V];
40
41     for(int remove = 0; remove < V; remove++){
42         {
43             for(int i=0;i<V;i++)
44                 for(int j=0;j<V;j++)
45                     temp[i][j]=graph[i][j];
46
47             for(int i=0;i<V;i++)
48             {
49                 temp[remove][i]=0;
50                 temp[i][remove]=0;
51             }
52
53             int visited[V]={0};
54
55             int start=0;
56             while(start<V && start==remove)
57                 start++;
58
59             DFS(temp,visited,start);
60
61             for(int i=0;i<V;i++)
62             {
63                 if(i!=remove && !visited[i])
64                     return 1;
65             }
66         }
67     }
68 }
```

```

66     }
67
68     return 2;
69 }
70
71 int edgeConnectivity(int graph[V][V])
72 {
73     int temp[V][V];
74
75     for(int i=0;i<V;i++)
76     {
77         for(int j=i+1;j<V;j++)
78         {
79             if(graph[i][j])
80             {
81                 for(int x=0;x<V;x++)
82                     for(int y=0;y<V;y++)
83                         temp[x][y]=graph[x][y];
84
85                 temp[i][j]=0;
86                 temp[j][i]=0;
87
88                 if(!isConnected(temp))
89                     return 1;
90             }
91         }
92     }
93
94     return 2;
95 }
96
97 int main()
98 {
99     int graph[V][V]={
100         {0,1,1,0,0,0},
101         {1,0,1,1,0,0},
102         {1,1,0,1,1,0},
103         {0,1,1,0,1,1},
104         {0,0,1,1,0,1},
105         {0,0,0,1,1,0}
106     };
107
108     if(isConnected(graph))
109         printf("Graph is Connected\n");
110     else
111         printf("Graph is Not Connected\n");
112
113     printf("Vertex Connectivity = %d\n",vertexConnectivity(graph));
114     printf("Edge Connectivity = %d\n",edgeConnectivity(graph));
115
116     if(vertexConnectivity(graph)==1)
117         printf("Graph is Separable\n");
118     else
119         printf("Graph is Non-Separable\n");
120
121     return 0;
122 }

```



```

1 // Objective: To determine the dominating set and domination number of a graph.
2 // Problem: Write a program to:
3 // 1. Find dominating sets
4 // 2. Find domination number
5 // 3. Find total dominating sets
6 // 4. Find total domination number
7 // Input: N = 4, M = 4, Edges = {(1,2), (1,3), (3,4), (2,4)}
8 // Output:
9 // Possible Dominating Sets:
10 // {1,4}
11 // {2,3}
12 // {1,2}
13 // {1,3}
14 // {2,4}
15 // {3,4}
16 // Domination Number = 2
17 // Total Dominating Sets: {1,3} {2,4}
18 // Total Domination Number = 2
19 #include <stdio.h>
20
21 #define N 4
22
23 int graph[N][N] = {
24     {0,1,1,0},
25     {1,0,0,1},
26     {1,0,0,1},
27     {0,1,1,0}
28 };
29
30 int countBits(int mask){
31     int c=0;
32     while(mask){
33         c += mask & 1;
34         mask >>=1;
35     }
36     return c;
37 }
38
39 int isDominating(int mask){
40     for(int v=0; v<N; v++){
41         if(mask & (1<<v)) continue;
42
43         int covered=0;
44         for(int u=0; u<N; u++){
45             if((mask&(1<<u)) && graph[u][v]){
46                 covered=1;
47                 break;
48             }
49         }
50         if(!covered) return 0;
51     }
52     return 1;
53 }
54
55 int isTotalDominating(int mask){
56     for(int v=0; v<N; v++){
57         int covered=0;
58         for(int u=0; u<N; u++){
59             if((mask&(1<<u)) && graph[u][v]){
60                 covered=1;
61                 break;
62             }
63         }
64         if(!covered) return 0;
65     }

```

```

66     return 1;
67 }
68
69 void printSet(int mask){
70     printf("{");
71     int first=1;
72     for(int i=0;i<N;i++){
73         if(mask&(1<<i)){
74             if(!first) printf(",");
75             printf("%d",i+1);
76             first=0;
77         }
78     }
79     printf("}");
80 }
81
82 int main(){
83     int minDom=N+1, minTotal=N+1;
84
85     printf("Possible Dominating Sets:\n");
86     for(int mask=1; mask<(1<<N); mask++){
87         if(isDominating(mask)){
88             printSet(mask);
89             printf("\n");
90             int sz=countBits(mask);
91             if(sz<minDom) minDom=sz;
92         }
93     }
94
95     printf("\nDomination Number = %d\n",minDom);
96
97     printf("Total Dominating Sets:\n");
98     for(int mask=1; mask<(1<<N); mask++){
99         if(isTotalDominating(mask)){
100             printSet(mask);
101             printf("\n");
102             int sz=countBits(mask);
103             if(sz<minTotal) minTotal=sz;
104         }
105     }
106
107     printf("\nTotal Domination Number = %d\n",minTotal);
108
109     return 0;
110 }

```

```

1 // Objective: To determine the total dominating set and total domination number of a graph.
2 // Problem: Write a program to:
3 // 1. Find total dominating sets
4 // 2. Find total domination number
5 // Input: N = 4, M = 4, Edges = {(1,2), (1,3), (3,4), (2,4)}
6 // Output:
7 // Total Dominating Sets: {1,3} {2,4}
8 // Total Domination Number = 2
9 #include <stdio.h>
10
11 #define N 4
12
13 int graph[N][N] = {
14     {0,1,1,0},
15     {1,0,0,1},
16     {1,0,0,1},
17     {0,1,1,0}
18 };
19
20 int countBits(int mask){
21     int c=0;
22     while(mask){
23         c += mask & 1;
24         mask >>=1;
25     }
26     return c;
27 }
28
29 int isTotalDominating(int mask){
30     for(int v=0; v<N; v++){
31         int covered=0;
32         for(int u=0; u<N; u++){
33             if((mask&(1<<u)) && graph[u][v]){
34                 covered=1;
35                 break;
36             }
37         }
38         if(!covered) return 0;
39     }
40     return 1;
41 }
42
43 void printSet(int mask){
44     printf("{");
45     int first=1;
46     for(int i=0;i<N;i++){
47         if(mask&(1<<i)){
48             if(!first) printf(",");
49             printf("%d",i+1);
50             first=0;
51         }
52     }
53     printf("}");
54 }
55
56 int main(){
57     int min=N+1;
58
59     printf("Total Dominating Sets:\n");
60
61     for(int mask=1; mask<(1<<N); mask++){
62         if(isTotalDominating(mask)){
63             printSet(mask);
64             printf("\n");
65

```

```
66         int size=countBits(mask);
67         if(size<min)
68             min=size;
69     }
70 }
71
72 printf("\nTotal Domination Number = %d\n",min);
73
74 return 0;
75 }
```

```
1 // Objective: To determine different types of matching in graphs
2 // Problem: Write a program to determine: Maximal matching, Maximum matching, and Perfect matching
3 // Input: Bipartite graph.
4 // Output:
5 // Matching sets and type of matching.
6 #include <stdio.h>
7
8 #define E 5
9 #define V 6
10
11 int edges[][2] = {
12     {0, 3},
13     {0, 4},
14     {1, 4},
15     {1, 5},
16     {2, 5}
17 };
18
19 int combinations[32][5] = {
20     {-1,-1,-1,-1,-1},
21
22     {0,-1,-1,-1,-1},
23     {1,-1,-1,-1,-1},
24     {2,-1,-1,-1,-1},
25     {3,-1,-1,-1,-1},
26     {4,-1,-1,-1,-1},
27
28     {0,1,-1,-1,-1},
29     {0,2,-1,-1,-1},
30     {0,3,-1,-1,-1},
31     {0,4,-1,-1,-1},
32     {1,2,-1,-1,-1},
33     {1,3,-1,-1,-1},
34     {1,4,-1,-1,-1},
35     {2,3,-1,-1,-1},
36     {2,4,-1,-1,-1},
37     {3,4,-1,-1,-1},
38
39     {0,1,2,-1,-1},
40     {0,1,3,-1,-1},
41     {0,1,4,-1,-1},
42     {0,2,3,-1,-1},
43     {0,2,4,-1,-1},
44     {0,3,4,-1,-1},
45     {1,2,3,-1,-1},
46     {1,2,4,-1,-1},
47     {1,3,4,-1,-1},
48     {2,3,4,-1,-1},
49
50     {0,1,2,3,-1},
51     {0,1,2,4,-1},
52     {0,1,3,4,-1},
53     {0,2,3,4,-1},
54     {1,2,3,4,-1},
55
56     {0,1,2,3,4}
57 };
58
59 int arr[32];
60 int p = 0;
61
62
63 void printMatching(int index)
64 {
65     int j;
```

```

66
67     printf("{ ");
68
69     for(j = 0; j < E; j++)
70     {
71         if(combinations[index][j] != -1)
72         {
73             printf("%d ", combinations[index][j]);
74         }
75     }
76
77     printf("}");
78 }
79
80
81 void findMatching()
82 {
83     int i, j, k;
84     int x, start, end;
85     int flag;
86     int freq[V];
87
88     printf("\n===== ALL MATCHINGS =====\n");
89
90     p = 0;
91
92     for(i = 0; i < 32; i++)
93     {
94         for(k = 0; k < V; k++)
95         {
96             freq[k] = 0;
97         }
98
99         flag = 1;
100
101         for(j = 0; j < E; j++)
102         {
103             x = combinations[i][j];
104
105             if(x == -1)
106                 continue;
107
108             start = edges[x][0];
109             end = edges[x][1];
110
111             if(freq[start] == 1 || freq[end] == 1)
112             {
113                 flag = 0;
114                 break;
115             }
116
117             freq[start] = 1;
118             freq[end] = 1;
119         }
120
121         if(flag == 1)
122         {
123             arr[p] = i;
124             p++;
125
126             printMatching(i);
127             printf("\n");
128         }
129     }
130
131     printf("\nTotal Matchings = %d\n", p);
132 }

```

```

133
134
135 void findMaximalMatching()
136 {
137     int i, j, e;
138     int index;
139     int edgeIndex;
140     int start, end;
141     int freq[V];
142     int maximal;
143
144     printf("\n===== MAXIMAL MATCHINGS =====\n");
145
146     for(i = 0; i < p; i++)
147     {
148         index = arr[i];
149
150         for(j = 0; j < V; j++)
151         {
152             freq[j] = 0;
153         }
154
155         for(j = 0; j < E; j++)
156         {
157             edgeIndex = combinations[index][j];
158
159             if(edgeIndex == -1)
160                 continue;
161
162             start = edges[edgeIndex][0];
163             end = edges[edgeIndex][1];
164
165             freq[start] = 1;
166             freq[end] = 1;
167         }
168
169         maximal = 1;
170
171         for(e = 0; e < E; e++)
172         {
173             int alreadyPresent = 0;
174
175             for(j = 0; j < E; j++)
176             {
177                 if(combinations[index][j] == e)
178                 {
179                     alreadyPresent = 1;
180                     break;
181                 }
182             }
183
184             if(alreadyPresent)
185                 continue;
186
187             start = edges[e][0];
188             end = edges[e][1];
189
190             if(freq[start] == 0 && freq[end] == 0)
191             {
192                 maximal = 0;
193                 break;
194             }
195         }
196
197         if(maximal == 1)
198         {
199             printMatching(index);

```

```

200         printf("\n");
201     }
202 }
203 }
204
205 void findMaximumMatching()
206 {
207     int i, j;
208     int index;
209     int size;
210     int maxSize = 0;
211
212     printf("\n===== MAXIMUM MATCHING =====\n");
213
214     for(i = 0; i < p; i++)
215     {
216         index = arr[i];
217
218         size = 0;
219
220         for(j = 0; j < E; j++)
221         {
222             if(combinations[index][j] != -1)
223             {
224                 size++;
225             }
226         }
227
228         if(size > maxSize)
229         {
230             maxSize = size;
231         }
232     }
233
234     for(i = 0; i < p; i++)
235     {
236         index = arr[i];
237
238         size = 0;
239
240         for(j = 0; j < E; j++)
241         {
242             if(combinations[index][j] != -1)
243             {
244                 size++;
245             }
246         }
247
248         if(size == maxSize)
249         {
250             printMatching(index);
251             printf("\n");
252         }
253     }
254
255     printf("\nMaximum Matching Size = %d\n", maxSize);
256 }
257
258
259 int main()
260 {
261     findMatching();
262
263     findMaximalMatching();
264
265     findMaximumMatching();
266

```



```
267     return 0;  
268 }
```

```
1 // Objective: Verify Hall's Condition
2 // Problem: Write a program to check whether Hall's condition holds for a given bipartite graph.
3 // Input: Bipartite graph with sets X and Y.
4 // Output:
5 // Hall's condition satisfied or not satisfied.
6 #include <stdio.h>
7
8 int graph[4][4] = {
9     {1, 1, 0, 0},
10    {1, 0, 1, 0},
11    {0, 1, 0, 1},
12    {0, 0, 1, 1}
13 };
14
15 int subsets[15][4] = {
16     {1,0,0,0},
17     {0,1,0,0},
18     {0,0,1,0},
19     {0,0,0,1},
20
21     {1,1,0,0},
22     {1,0,1,0},
23     {1,0,0,1},
24     {0,1,1,0},
25     {0,1,0,1},
26     {0,0,1,1},
27
28     {1,1,1,0},
29     {1,1,0,1},
30     {1,0,1,1},
31     {0,1,1,1},
32
33     {1,1,1,1}
34 };
35
36 int checkHallsCondition()
37 {
38     int i, j, k;
39     int leftCount, rightCount;
40     int used[4];
41
42     for(i = 0; i < 15; i++)
43     {
44         leftCount = 0;
45         rightCount = 0;
46
47         // used array ne 0 thi initialize karo
48         for(j = 0; j < 4; j++)
49             used[j] = 0;
50
51         // Count vertices in subset
52         for(j = 0; j < 4; j++)
53         {
54             if(subsets[i][j] == 1)
55             {
56                 leftCount++;
57
58                 // Find neighbours
59                 for(k = 0; k < 4; k++)
60                 {
61                     if(graph[j][k] == 1 && used[k] == 0)
62                     {
63                         used[k] = 1;
64                         rightCount++;
65                     }
66                 }
67             }
68         }
69     }
70 }
```

```
66         }
67     }
68 }
69
70     // Hall's Condition Check
71     if(rightCount < leftCount)
72         return 0;
73 }
74
75     return 1;
76 }
77
78 void main()
79 {
80     int ans;
81
82     ans = checkHallsCondition();
83
84     if(ans == 1)
85         printf("Hall's condition is satisfied");
86     else
87         printf("Hall's condition is not satisfied");
88 }
```

```
1 // Objective: To construct different matrices associated with graphs.
2 // Problem: Write a program to construct: Adjacency Matrix, Degree Matrix, and Laplacian Matrix
3 // Input: N = 5, M = 4, Edges = {(1,2), (2,3), (4,5), (1,5)}
4 // Output
5 // Adjacency, Degree, and Laplacian matrices.
6 #include <stdio.h>
7
8 #define N 5
9
10 int main() {
11     int adj[N][N] = {0};
12     int degree[N][N] = {0};
13     int laplacian[N][N];
14
15     int edges[4][2] = {
16         {1,2}, {2,3}, {4,5}, {1,5}
17     };
18
19     for(int i=0; i<N-1; i++) {
20         int u = edges[i][0] - 1;
21         int v = edges[i][1] - 1;
22         adj[u][v] = 1;
23         adj[v][u] = 1;
24     }
25
26     for(int i=0; i<N; i++) {
27         int deg = 0;
28         for(int j=0; j<N; j++)
29             deg += adj[i][j];
30         degree[i][i] = deg;
31     }
32
33     for(int i=0; i<N; i++)
34         for(int j=0; j<N; j++)
35             laplacian[i][j] = degree[i][j] - adj[i][j];
36
37     printf("Adjacency Matrix:\n");
38     for(int i=0; i<N; i++) {
39         for(int j=0; j<N; j++)
40             printf("%d ", adj[i][j]);
41         printf("\n");
42     }
43
44     printf("\nDegree Matrix:\n");
45     for(int i=0; i<N; i++) {
46         for(int j=0; j<N; j++)
47             printf("%d ", degree[i][j]);
48         printf("\n");
49     }
50
51     printf("\nLaplacian Matrix:\n");
52     for(int i=0; i<N; i++) {
53         for(int j=0; j<N; j++)
54             printf("%d ", laplacian[i][j]);
55         printf("\n");
56     }
57
58     return 0;
59 }
```

```

1 // Objective: To compute eigenvalues and eigenvectors of graph matrices.
2 // Problem: Write a program to calculate eigenvalues and eigenvectors of adjacency and Laplacian mat
3 // Input: Adjacency or Laplacian matrix.
4 // Output:
5 // Eigenvalues and corresponding eigenvectors.
6 // 13 Objective: To compute eigenvalues and eigenvectors of graph matrices.
7 // Problem: Write a program to calculate eigenvalues and eigenvectors of adjacency and Laplacian mat
8 // Input: Adjacency or Laplacian matrix.
9 // Output:
10 // Eigenvalues and corresponding eigenvectors.
11
12 #include <stdio.h>
13 #include <math.h>
14
15 #define N 3
16
17 int main()
18 {
19     int i, j, k;
20     double A[N][N] = {
21         {0, 1, 1},
22         {1, 0, 1},
23         {1, 1, 0}
24     };
25
26     double V[N][N] = {0};
27
28     for (i = 0; i < N; i++)
29         V[i][i] = 1;
30
31     for (k = 0; k < 100; k++)
32     {
33         int p = 0, q = 1;
34
35         for (i = 0; i < N; i++)
36             for (j = i + 1; j < N; j++)
37                 if (fabs(A[i][j]) > fabs(A[p][q]))
38                     p = i, q = j;
39
40         if (fabs(A[p][q]) < 0.000001)
41             break;
42
43         double theta = 0.5 * atan2(2 * A[p][q],
44                                     A[p][p] - A[q][q]);
45
46         double c = cos(theta);
47         double s = sin(theta);
48
49         for (i = 0; i < N; i++)
50         {
51             double x = A[i][p];
52             double y = A[i][q];
53
54             A[i][p] = c * x - s * y;
55             A[i][q] = s * x + c * y;
56         }
57
58         for (i = 0; i < N; i++)
59         {
60             double x = A[p][i];
61             double y = A[q][i];
62
63             A[p][i] = c * x - s * y;
64             A[q][i] = s * x + c * y;
65         }
66     }
67 }

```

```
66
67     for (i = 0; i < N; i++)
68     {
69         double x = V[i][p];
70         double y = V[i][q];
71
72         V[i][p] = c * x - s * y;
73         V[i][q] = s * x + c * y;
74     }
75 }
76
77 printf("Eigenvalues:\n");
78 for (i = 0; i < N; i++)
79     printf("%.2lf ", A[i][i]);
80
81 printf("\n\nEigenvectors:\n");
82 for (i = 0; i < N; i++)
83 {
84     for (j = 0; j < N; j++)
85         printf("%.2lf ", V[i][j]);
86     printf("\n");
87 }
88
89 return 0;
90 }
```

```

1 // Objective: To perform spectral clustering on a graph using the Laplacian matrix and eigenvectors
2 // Problem: Write a program to perform spectral clustering on an undirected graph by using the graph
3 // The program should:
4 // 1. Construct the adjacency matrix of the graph.
5 // 2. Compute the degree matrix.
6 // 3. Construct the Laplacian matrix.
7 // 4. Compute eigenvalues and eigenvectors of the Laplacian matrix.
8 // 5. Partition the graph into clusters using spectral properties.
9 // Input:
10 // N = 6
11 // Edges = {(0,1),(0,2),(1,2),(3,4),(4,5),(3,5)}
12 // Output:
13 // Cluster 1: {0,1,2}
14 // Cluster 2: {3,4,5}
15 #include <stdio.h>
16 #include <math.h>
17
18 #define N 6
19
20 void multiply(double A[N][N], double x[N], double y[N])
21 {
22     for (int i = 0; i < N; i++)
23     {
24         y[i] = 0;
25         for (int j = 0; j < N; j++)
26             y[i] += A[i][j] * x[j];
27     }
28 }
29
30 void normalize(double x[N])
31 {
32     double norm = 0;
33
34     for (int i = 0; i < N; i++)
35         norm += x[i] * x[i];
36
37     norm = sqrt(norm);
38
39     for (int i = 0; i < N; i++)
40         x[i] /= norm;
41 }
42
43 int main()
44 {
45     int adjacency[N][N] = {0};
46     int degree[N][N] = {0};
47     double L[N][N] = {0};
48
49     int edges[6][2] = {
50         {0,1}, {0,2}, {1,2},
51         {3,4}, {4,5}, {3,5}
52     };
53
54     for (int i = 0; i < 6; i++)
55     {
56         int u = edges[i][0];
57         int v = edges[i][1];
58
59         adjacency[u][v] = 1;
60         adjacency[v][u] = 1;
61     }
62
63     for (int i = 0; i < N; i++)
64     {
65         int d = 0;

```

```

66
67     for (int j = 0; j < N; j++)
68         d += adjacency[i][j];
69
70     degree[i][i] = d;
71 }
72
73 for (int i = 0; i < N; i++)
74 {
75     for (int j = 0; j < N; j++)
76         L[i][j] = degree[i][j] - adjacency[i][j];
77 }
78
79 printf("Adjacency Matrix:\n");
80 for (int i = 0; i < N; i++)
81 {
82     for (int j = 0; j < N; j++)
83         printf("%d ", adjacency[i][j]);
84     printf("\n");
85 }
86
87 printf("\nDegree Matrix:\n");
88 for (int i = 0; i < N; i++)
89 {
90     for (int j = 0; j < N; j++)
91         printf("%d ", degree[i][j]);
92     printf("\n");
93 }
94
95 printf("\nLaplacian Matrix:\n");
96 for (int i = 0; i < N; i++)
97 {
98     for (int j = 0; j < N; j++)
99         printf("%.0f ", L[i][j]);
100     printf("\n");
101 }
102
103 double x[N] = {1, -1, 1, -1, 1, -1};
104 double y[N];
105
106 for (int iter = 0; iter < 20; iter++)
107 {
108     multiply(L, x, y);
109
110     double mean = 0;
111
112     for (int i = 0; i < N; i++)
113         mean += y[i];
114
115     mean /= N;
116
117     for (int i = 0; i < N; i++)
118         x[i] = y[i] - mean;
119
120     normalize(x);
121 }
122
123 printf("\nFiedler Vector:\n");
124 for (int i = 0; i < N; i++)
125     printf("Vertex %d : %.3f\n", i, x[i]);
126
127 printf("\nCluster 1: { ");
128 for (int i = 0; i < N; i++)
129     if (x[i] >= 0)
130         printf("%d ", i);
131
132 printf("}\nCluster 2: { ");

```



```
133
134     for (int i = 0; i < N; i++)
135         if (x[i] < 0)
136             printf("%d ", i);
137
138     printf("}\n");
139
140     return 0;
141 }
```

```
1 // Objective: To find shortest paths between all pairs of vertices.
2 // Problem: Write a program to implement Floyd-Warshall Algorithm.
3 // Input:
4 // Weighted graph represented using adjacency matrix.
5 // Output:
6 // Shortest path matrix between all pairs of vertices.
7 #include <stdio.h>
8
9 #define N 4
10 #define INF 99999
11
12 int main()
13 {
14     int graph[N][N] = {
15         {0, 5, INF, 10},
16         {INF, 0, 3, INF},
17         {INF, INF, 0, 1},
18         {INF, INF, INF, 0}
19     };
20
21     int dist[N][N];
22
23     for (int i = 0; i < N; i++)
24         for (int j = 0; j < N; j++)
25             dist[i][j] = graph[i][j];
26
27     for (int k = 0; k < N; k++)
28         for (int i = 0; i < N; i++)
29             for (int j = 0; j < N; j++)
30                 if (dist[i][k] != INF && dist[k][j] != INF &&
31                     dist[i][k] + dist[k][j] < dist[i][j])
32                     dist[i][j] = dist[i][k] + dist[k][j];
33
34     printf("Shortest Path Matrix:\n\n");
35
36     for (int i = 0; i < N; i++)
37     {
38         for (int j = 0; j < N; j++)
39         {
40             if (dist[i][j] == INF)
41                 printf("INF ");
42             else
43                 printf("%3d ", dist[i][j]);
44         }
45         printf("\n");
46     }
47
48     return 0;
49 }
```